NYCU 2026 Spring Deep Learning Lab6
Conditional Generative Models

謝濬遠
Student ID: 114024511

1 Introduction

In this lab, we implement conditional generative models for the i-CLEVR image generation task. Given a multi-label condition, such as

`["red sphere", "cyan cylinder", "cyan cube"]`,

the model is required to generate a synthetic image containing the corresponding objects.

We implemented and compared four sampling or generation settings:

- DDPM with linear noise schedule
- DDPM with cosine noise schedule
- DDIM sampling based on the DDPM linear model
- Conditional Flow Matching

The generated images are evaluated using the TA-provided pretrained evaluator. The main evaluation metrics are the classification accuracy on `test.json`, the classification accuracy on `new_test.json`, the mean accuracy, and the total sampling time.

2 Implementation Details

2.1 Dataset and Label Encoding

The dataset used in this lab is the i-CLEVR dataset. Each image contains one or more colored objects. The object categories are defined in `object.json`, which contains 24 classes.

Each label set is converted into a 24-dimensional multi-hot vector. For example,

$$[\texttt{red sphere}, \texttt{cyan cube}] \rightarrow [0, 1, 0, \cdots, 1, 0]. \tag{1}$$

All images are resized to 64×64 and normalized into $[-1, 1]$ by

$$x_{\text{norm}} = 2x - 1. \tag{2}$$

2.2 Conditional UNet Architecture

All generative models in this experiment use the same conditional UNet backbone. The model takes an image state $x_t \in \mathbb{R}^{B\times3\times64\times64}$, a time step t, and a multi-label condition vector $y \in \mathbb{R}^{B\times24}$ as input. The output has the same spatial size as the input image. For DDPM and DDIM, the output is the predicted noise $\epsilon_\theta(x_t, t, y)$; for Flow Matching, the output is the predicted velocity field $v_\theta(x_t, t, y)$.

The overall architecture is shown in Figure 1. The network follows an encoder-decoder UNet structure with three downsampling blocks, two bottleneck residual blocks, and three upsampling blocks. Skip connections are used to concatenate encoder features with decoder features at the same spatial resolution.

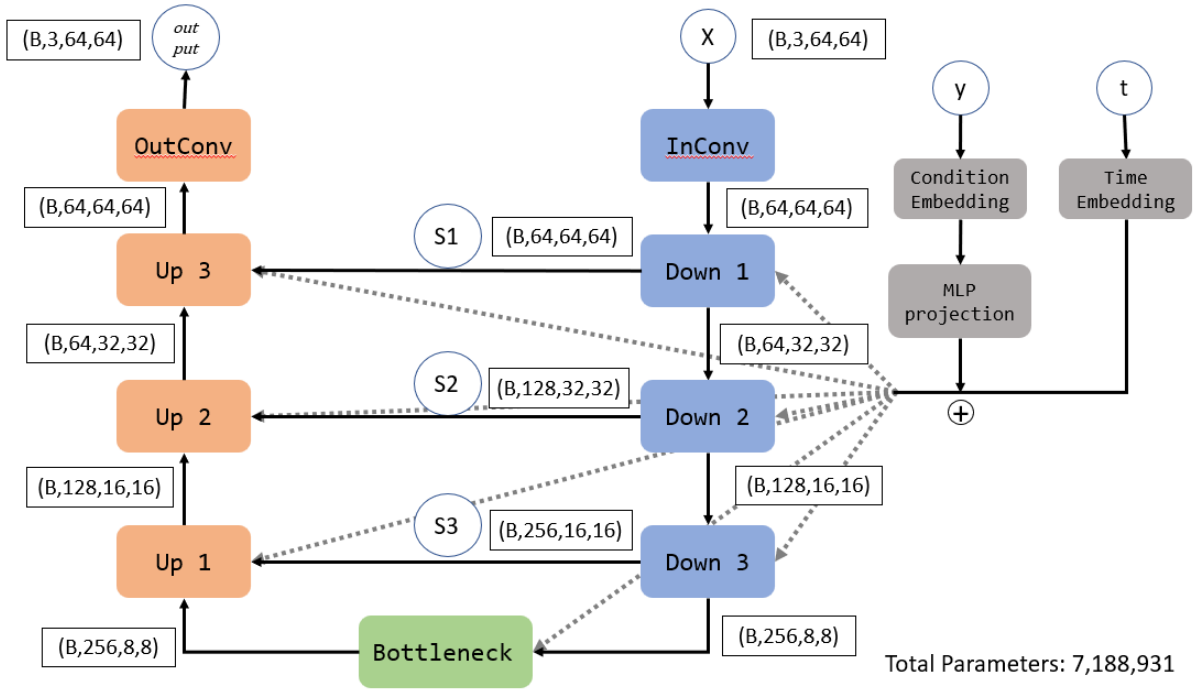

Figure 1: Conditional UNet architecture. The condition embedding and time embedding are added together and injected into each residual block.

2.2.1 Time and Condition Embedding

The time step t is first converted into a sinusoidal positional embedding. For embedding dimension d, the sinusoidal time embedding is defined as

$$\mathrm{TE}(t) = \left[\sin(t\omega_1), \ldots, \sin(t\omega_{d/2}), \cos(t\omega_1), \ldots, \cos(t\omega_{d/2})\right], \tag{3}$$

where ω_i denotes the frequency term. The sinusoidal embedding is then passed through an MLP:

$$e_t = \mathrm{MLP}_t(\mathrm{TE}(t)). \tag{4}$$

The multi-label condition vector y is also projected by an MLP:

$$e_y = \mathrm{MLP}_y(y). \tag{5}$$

The final conditioning vector is obtained by adding the two embeddings:

$$e = e_t + e_y. \tag{6}$$

This combined embedding e is injected into every residual block of the UNet.

2.2.2 Residual Block

Each downsampling block, upsampling block, and bottleneck block contains residual convolutional blocks. Given an input feature map h, the residual block first applies a 3×3 convolution, group normalization, and SiLU activation:

$$h_1 = \mathrm{SiLU}(\mathrm{GN}(\mathrm{Conv}_{3\times3}(h))). \tag{7}$$

The conditioning vector e is projected to the channel dimension of the feature map and added to h_1:

$$h_1' = h_1 + W_e e, \tag{8}$$

where W_e is a linear projection layer.

Then, another 3×3 convolution, group normalization, and SiLU activation are applied:

$$h_2 = \text{SiLU}(\text{GN}(\text{Conv}_{3\times3}(h_1'))). \tag{9}$$

Finally, a residual connection is added:

$$\text{ResBlock}(h, e) = h_2 + \text{Shortcut}(h). \tag{10}$$

If the input and output channel dimensions are different, the shortcut branch uses a 1×1 convolution to match the channel dimension. Otherwise, it uses an identity mapping.

2.2.3 Downsampling Block

Each downsampling block consists of one residual block followed by a strided convolution for spatial downsampling:

$$s_i = \text{ResBlock}(h_i, e), \tag{11}$$

$$h_{i+1} = \text{Conv}_{4\times4,s=2}(s_i). \tag{12}$$

Here, s_i is stored as the skip feature and later concatenated with the corresponding decoder feature. The spatial resolution is reduced by half after each downsampling block.

The feature sizes in the encoder are:

$$64 \times 64 \to 32 \times 32 \to 16 \times 16 \to 8 \times 8. \tag{13}$$

In this implementation, the channel dimensions are:

$$64 \to 64 \to 128 \to 256. \tag{14}$$

2.2.4 Bottleneck

The bottleneck contains two residual blocks:

$$h_{\text{mid}} = \text{ResBlock}_2(\text{ResBlock}_1(h_3, e), e). \tag{15}$$

This part processes the lowest-resolution feature map with the largest number of channels.

2.2.5 Upsampling Block

Each upsampling block first applies a transposed convolution to increase the spatial resolution:

$$\tilde{h}_i = \text{ConvTranspose}_{4\times4,s=2}(h_i). \tag{16}$$

Then, the upsampled feature map is concatenated with the corresponding skip feature from the encoder:

$$h_i^{cat} = \text{Concat}(\tilde{h}_i, s_i). \tag{17}$$

After concatenation, a residual block is applied:

$$h_{i-1} = \text{ResBlock}(h_i^{cat}, e). \tag{18}$$

The decoder gradually restores the spatial resolution:

$$8 \times 8 \to 16 \times 16 \to 32 \times 32 \to 64 \times 64. \tag{19}$$

2.2.6 Output Layer

The final output layer consists of group normalization, SiLU activation, and a 3×3 convolution:

$$\hat{o} = \mathrm{Conv}_{3\times3}(\mathrm{SiLU}(\mathrm{GN}(h))). \tag{20}$$

For DDPM and DDIM, the output is interpreted as:

$$\hat{o} = \epsilon_\theta(x_t, t, y). \tag{21}$$

For Flow Matching, the same output tensor is interpreted as:

$$\hat{o} = v_\theta(x_t, t, y). \tag{22}$$

2.3 DDPM

In DDPM, the forward diffusion process gradually adds Gaussian noise to the clean image x_0:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, \tag{23}$$

where $\epsilon \sim \mathcal{N}(0, I)$.

The model is trained to predict the added noise:

$$\epsilon_\theta(x_t, t, y). \tag{24}$$

The training objective is:

$$L_{\mathrm{DDPM}} = \mathbb{E}_{x_0,\epsilon,t}\left[\|\epsilon - \epsilon_\theta(x_t, t, y)\|^2\right]. \tag{25}$$

During sampling, the model starts from pure Gaussian noise and iteratively denoises the image.

2.4 DDIM

Denoising Diffusion Implicit Model (DDIM) is a faster sampling method derived from the DDPM framework. DDIM uses the same noise prediction model trained by the DDPM objective, but modifies the reverse sampling process. Therefore, no additional training is required for DDIM. In this experiment, DDIM sampling is applied to the best DDPM-linear checkpoint.

In DDPM, the reverse process is stochastic and usually requires a large number of denoising steps. In contrast, DDIM defines a non-Markovian reverse process that can generate samples using fewer time steps. This makes DDIM much faster during inference.

Given a noisy image x_t and the predicted noise $\epsilon_\theta(x_t, t, y)$, the clean image can be estimated as:

$$\hat{x}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t}\epsilon_\theta(x_t, t, y)}{\sqrt{\bar{\alpha}_t}}. \tag{26}$$

Then, DDIM updates the image from time step t to a previous time step t' by:

$$x_{t'} = \sqrt{\bar{\alpha}_{t'}}\hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t'} - \sigma_t^2}\epsilon_\theta(x_t, t, y) + \sigma_t z, \tag{27}$$

where $z \sim \mathcal{N}(0, I)$, and σ_t controls the stochasticity of the sampling process.

The noise scale is defined as:

$$\sigma_t = \eta\sqrt{\frac{1 - \bar{\alpha}_{t'}}{1 - \bar{\alpha}_t}\left(1 - \frac{\bar{\alpha}_t}{\bar{\alpha}_{t'}}\right)}. \tag{28}$$

When $\eta = 0$, the sampling process becomes deterministic:

$$x_{t'} = \sqrt{\bar{\alpha}_{t'}}\hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t'}}\epsilon_\theta(x_t, t, y). \tag{29}$$

In this experiment, we use deterministic DDIM sampling with $\eta = 0$ and 50 sampling steps. Compared with DDPM, which uses 1000 reverse denoising steps, DDIM greatly reduces sampling time while maintaining reasonable image quality. This allows us to evaluate the trade-off between generation speed and evaluator accuracy.

2.5 Flow Matching

Flow Matching learns a continuous vector field between real data and Gaussian noise. Let x_0 be a clean image and $x_1 \sim \mathcal{N}(0, I)$ be Gaussian noise. The interpolated sample is defined as

$$x_t = (1 - t)x_0 + tx_1. \tag{30}$$

The target velocity field is

$$v_t = x_1 - x_0. \tag{31}$$

The model is trained to predict

$$v_\theta(x_t, t, y). \tag{32}$$

The Flow Matching loss is

$$L_{\text{FM}} = \mathbb{E}_{x_0, x_1, t} \left[\| v_t - v_\theta(x_t, t, y) \|^2 \right]. \tag{33}$$

During generation, the model starts from Gaussian noise and solves the reverse ODE. In this experiment, Flow Matching uses 100 sampling steps.

2.6 Training Settings

The hyperparameters are shown in Table 1. The only modified hyperparameter is the number of epochs, which is set to 1000. For DDIM, we use the same DDPM-linear trained model and only replace the sampling procedure. Therefore, the DDIM result reflects the effect of faster sampling rather than a separately trained model.

Table 1: Training settings.

Parameter	Value
Image size	64×64
Batch size	64
Optimizer	AdamW
Learning rate	1×10^{-4}
Epochs	1000
DDPM timesteps	1000
DDIM sampling steps	50
Flow Matching sampling steps	100
Base channels	64
Time embedding dimension	256
Condition embedding dimension	256
Gradient clipping	1.0

3 Experimental Results

3.1 Training Curves

Figure 2 shows the training loss curves of DDPM with linear schedule, DDPM with cosine schedule, and Flow Matching.

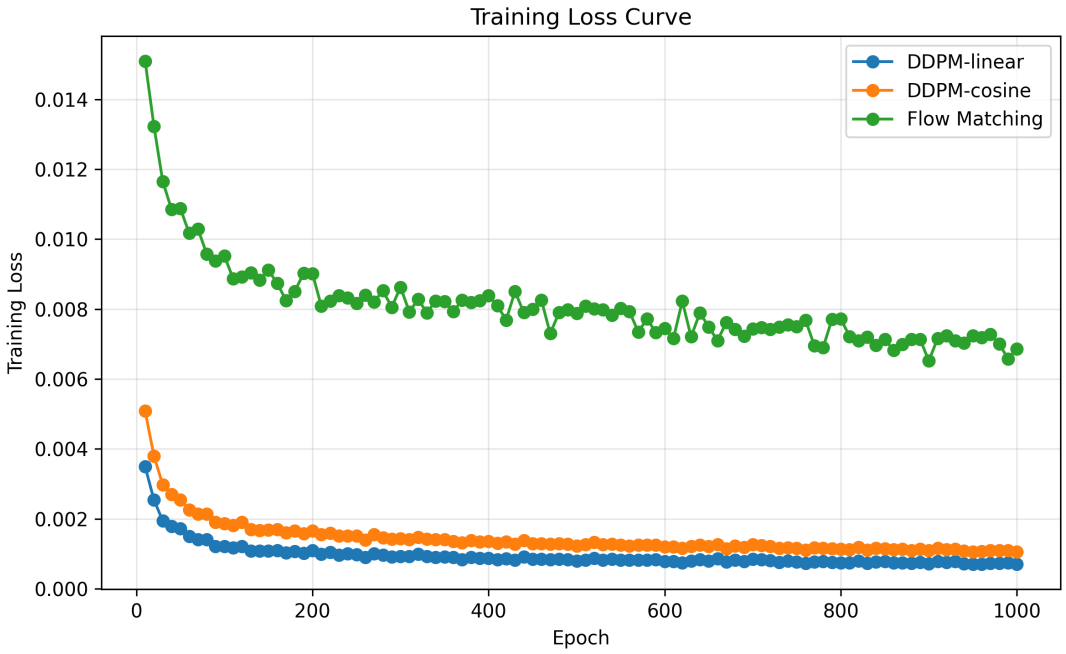

Figure 2: Training loss curves of DDPM-linear, DDPM-cosine, and Flow Matching.

Figure 3 shows the evaluator accuracy curves during training.

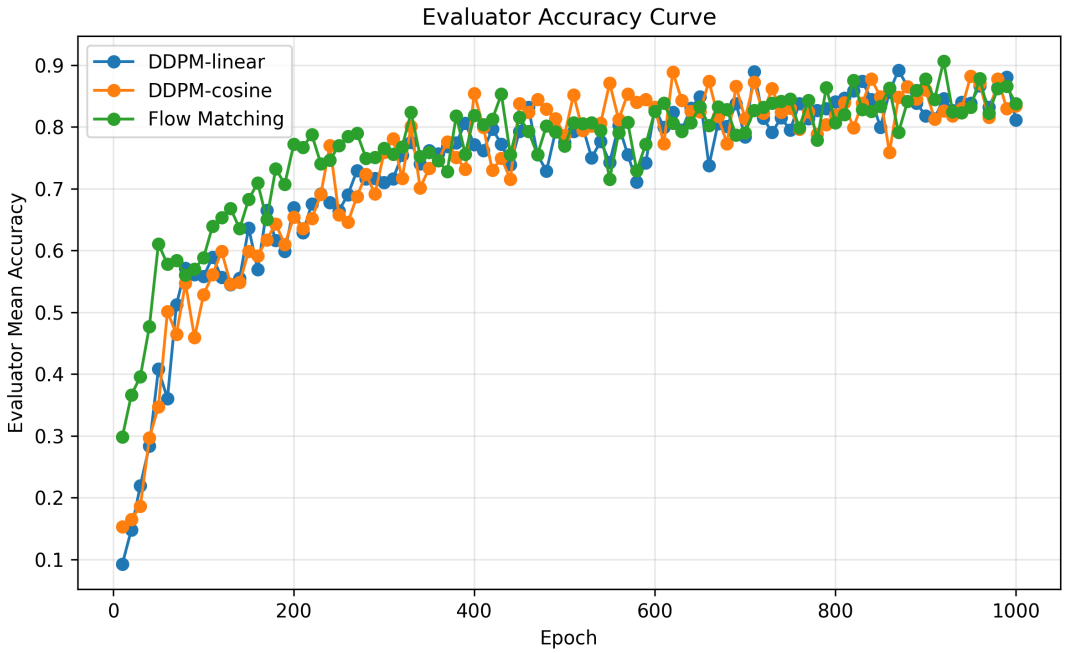

Figure 3: Evaluator accuracy curves of DDPM-linear, DDPM-cosine, and Flow Matching.

From Figure 2, all methods show a decreasing training loss during optimization. Flow Matching converges faster in the early training stage, while DDPM-based methods exhibit more stable optimization behavior.

From Figure 3, evaluator accuracy generally increases during training. Flow Matching achieves the highest evaluator accuracy among all methods.

3.2 Synthetic Image Grids

Figure 4 and Figure 5 show the generated image grids by ddpm-linear on `test.json` and `new_test.json`.

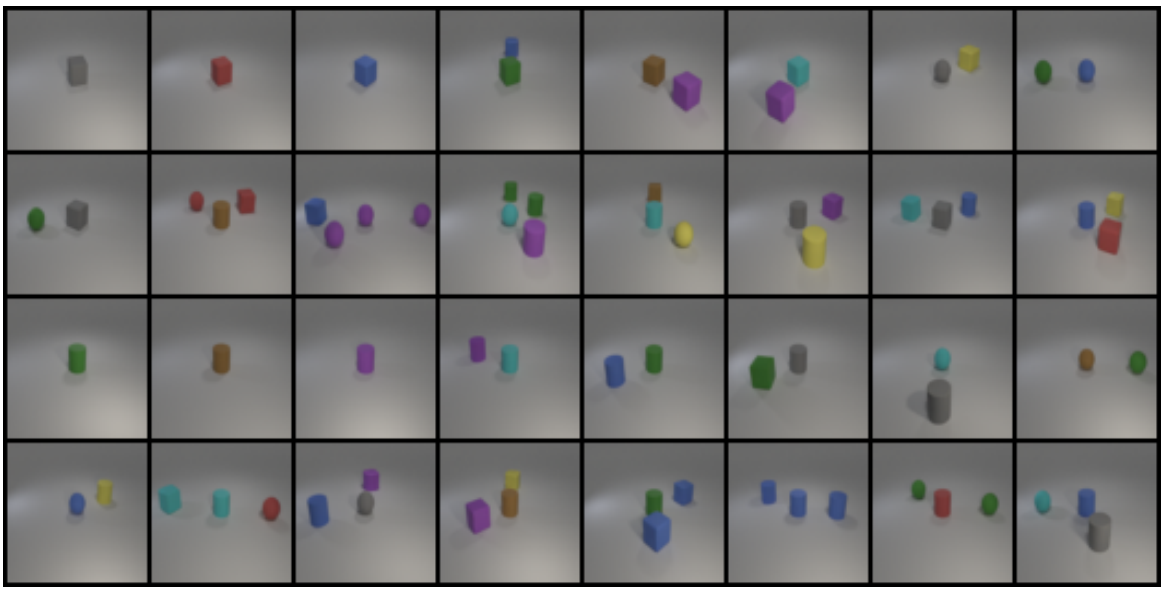

Figure 4: Synthetic image grid on `test.json`.

Figure 5: Synthetic image grid on `new_test.json`.

3.3 Denoising Process

Figure 7 shows the DDPM denoising process for the label set:

`["red sphere", "cyan cylinder", "cyan cube"]`.

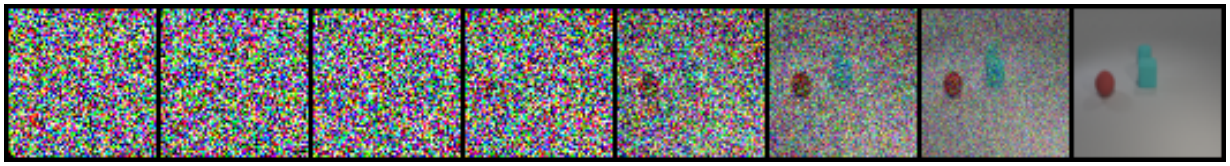

Figure 6: DDPM-linear denoising process from Gaussian noise to a generated image.

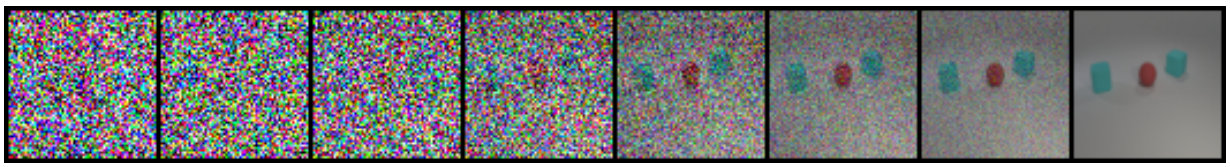

Figure 7: DDPM-cosine denoising process from Gaussian noise to a generated image.

3.4 Accuracy and Sampling Time

Table 2 summarizes the best experimental results selected from the evaluator accuracy curves. For DDPM-linear, DDPM-cosine, and Flow Matching, the reported accuracies are obtained from the checkpoints with the highest mean evaluator accuracy during training. For DDIM-linear, the result is obtained by applying DDIM sampling to the best DDPM-linear checkpoint. The mean accuracy is computed as the average of the accuracies on `test.json` and `new_test.json`.

Table 2: Best evaluator accuracy selected from the training accuracy curves.

Method	test.json Acc.	new_test.json Acc.	Mean Acc.	Sampling Time
DDPM-linear	0.902778	0.880885	0.891831	22.67 sec
DDPM-cosine	0.847222	0.917470	0.882346	22.66 sec
DDIM-linear	0.861111	0.869048	0.865079	1.59 sec
Flow Matching	0.930556	0.882019	0.906287	2.59 sec

Table 3 shows the number of sampling steps used by each method.

Table 3: Sampling time comparison on 64 generated images, including all images from `test.json` and `new_test.json`.

Method	Sampling Steps	Total Time for 64 Images	Sec / Image
DDPM-linear	1000	22.67 sec	0.3542
DDPM-cosine	1000	22.66 sec	0.3541
DDIM-linear	50	1.59 sec	0.0248
Flow Matching	100	2.59 sec	0.0405

3.5 Accuracy Screenshots

The evaluator outputs are shown in Figure 8.

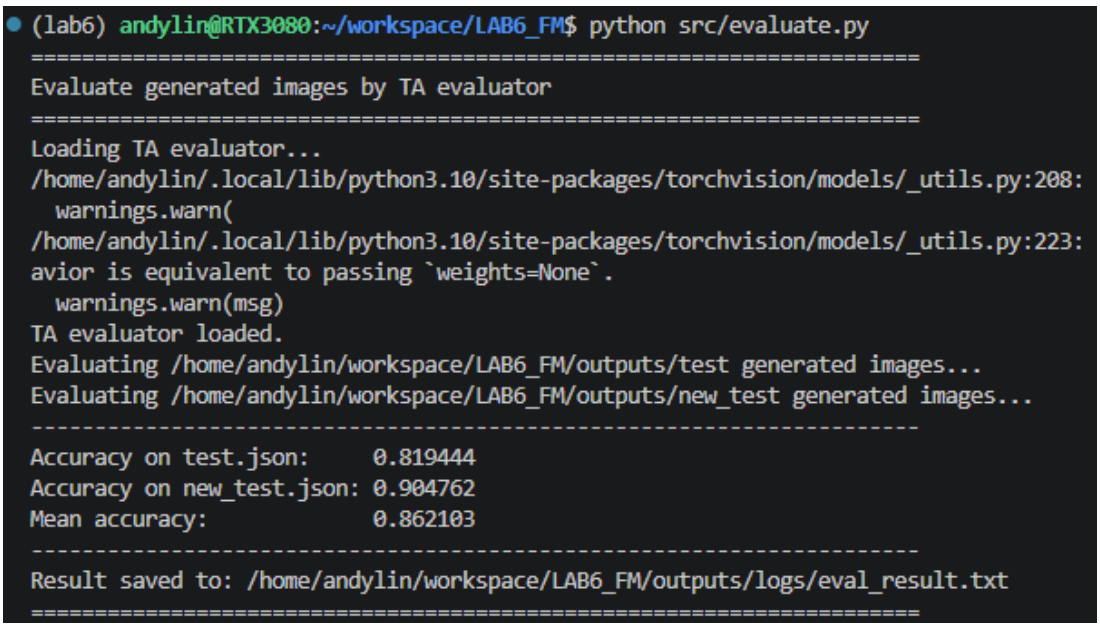

Figure 8: Accuracy screenshot(ddpm-linear) from the TA-provided evaluator.

4 Discussion

From Table 2, Flow Matching achieves the highest mean accuracy among all methods, with a mean accuracy of 0.906287. It also achieves the highest accuracy on `test.json`, reaching 0.930556. This suggests that Flow Matching can effectively learn the conditional generation task while maintaining fast sampling.

DDPM with linear noise schedule achieves a mean accuracy of 0.891831, which is slightly higher than DDPM with cosine noise schedule. Although the cosine schedule performs better on `new_test.json`, its performance on `test.json` is lower. This indicates that the cosine schedule may improve generalization to the new test conditions, but its overall performance is slightly less stable in this experiment.

DDIM-linear achieves the fastest sampling time, requiring only 1.59 seconds for the full sampling process. However, its mean accuracy is 0.865079, which is lower than both DDPM-linear and Flow Matching. This result shows the trade-off between sampling speed and generation quality. DDIM greatly accelerates sampling by reducing the number of sampling steps from 1000 to 50, but this speedup slightly decreases the evaluator accuracy.

Comparing generation time, DDPM-linear and DDPM-cosine both require around 22.6 seconds because both use 1000 reverse diffusion steps. Flow Matching only uses 100 steps and requires 2.59 seconds, while DDIM uses 50 steps and requires 1.59 seconds. Therefore, both DDIM and Flow Matching are much faster than standard DDPM.

Overall, Flow Matching provides the best balance between accuracy and sampling efficiency in this experiment. DDPM-linear remains a strong baseline with stable generation quality, while DDIM is the most efficient method when sampling speed is the main concern.

5 Conclusion

In this lab, we implemented and compared conditional DDPM, DDIM sampling, and conditional Flow Matching for multi-label image generation on the i-CLEVR dataset.

The experimental results show that Flow Matching achieves the best mean accuracy, while DDIM achieves the fastest sampling speed. DDPM provides stable generation quality but requires significantly more sampling time because of its iterative denoising process.

The comparison demonstrates that different generative modeling methods have different trade-offs. DDPM is stable but slow, DDIM is fast but slightly less accurate, and Flow Matching provides a strong balance between accuracy and efficiency.

References

1. Ho, Jonathan, Ajay Jain, and Pieter Abbeel. "Denoising Diffusion Probabilistic Models." NeurIPS, 2020.
2. Song, Jiaming, Chenlin Meng, and Stefano Ermon. "Denoising Diffusion Implicit Models." ICLR, 2021.
3. Lipman, Yaron, et al. "Flow Matching for Generative Modeling." ICLR, 2023.
4. Hugging Face Diffusion Models Course. `https://huggingface.co/learn/diffusion-course`
5. NYCU 2026 Spring Deep Learning Lab6.